ARM Assembly Fundamentals

Introduction to ARM Cortex-M Microprocessors — Chapter 5

Muním Zabidi

Faculty of Electrical Engineering, Universiti Teknologi Malaysia

Table of Contents

1. Why This Matters
2. How Much Assembly You Need
3. A First Assembly Program
4. The Load-Store Workflow
5. Memory Transfer Instructions
6. Optional: Signed Loads
7. The Core Register Set in Practice
8. Constants: `MOV`, `MVN`, and Building an Address
9. Worked Example: Reading Your First Disassembly
10. Summary

Why This Chapter Matters

Your C code reaches a hardware register, and that code compiles into instructions. At some time you must read them: in a fault handler, in unexpected compiler output, in an exact timing loop, or in startup code. The next three chapters make the machine visible. The goal is not fluency, but the ability to see what the processor does under any line of C.

After this chapter, you can:

1. Read an assembly program: directives, labels, instructions.
2. Apply load–process–store to any memory variable.
3. Choose the right load or store for byte, halfword and word; explain sign and zero extension.
4. Trace a short sequence register by register.
5. Explain low versus high registers and code size.
6. Recover the C statement from a short disassembly.

© Munim Zabidi

By the end of this chapter you can read what a debugger shows for `total = a + b;` and say where every value is, at each step:

```
LDR   R0, [R1]        ; load a
LDR   R2, [R1, #4]    ; load b
ADD   R0, R0, R2      ; add: registers only
STR   R0, [R1, #8]    ; store total
```

Questions this code raises, and where this chapter answers them:

Question raised	Answered in
Why four instructions, not one?	The load-store workflow
What do the brackets mean?	Memory transfer instructions
What if the data is a byte, not a word?	Byte and halfword transfers
How do I read it in a disassembly window?	A worked disassembly

Only one of the four is arithmetic.

© Munim Zabidi

BEAMER EXTRA

Table of Contents

1 Why This Matters

2 How Much Assembly You Need

3 A First Assembly Program

4 The Load-Store Workflow

5 Memory Transfer Instructions

6 Optional: Signed Loads

7 The Core Register Set in Practice

8 Constants: `MOV`, `MVN`, and Building an Address

9 Worked Example: Reading Your First Disassembly

10 Summary

Three Levels of Assembly Knowledge

The Cortex-M4 has about seventy instructions. This book asks you to know them at three different depths. Most of what you meet falls into the first two.

Level	What belongs here
Understand	`MOV`, `LDR`, `STR`, `ADD`, `SUB`, compare and branch, `PUSH`/`POP`. The core vocabulary of the next three chapters. Read, predict, and write.
Recognise	`MOVW`/`MOVT` pairs, addressing-mode forms, patterns a compiler prefers. Know them when you see them.
Know it exists	Saturating arithmetic, DSP/SIMD, coprocessor and floating-point, rare Thumb-2 forms. Look them up in Appendix C.

The goal of these three chapters

Not fluency, but literacy. You need enough skill to read a disassembly window and explain what the processor is doing.

Table of Contents

1 Why This Matters

2 How Much Assembly You Need

3 **A First Assembly Program**

4 The Load-Store Workflow

5 Memory Transfer Instructions

6 Optional: Signed Loads

7 The Core Register Set in Practice

8 Constants: `MOV`, `MVN`, and Building an Address

9 Worked Example: Reading Your First Disassembly

10 Summary

A Complete ARM Assembly Program

Something smaller than the blinking LED: add two constants, then stop. The shape of an assembly file — directives, labels, instructions — is visible before any register touches hardware.

```
; Define register aliases and constants
VAL1    RN  R1              ; VAL1 is an alias for R1
VAL2    RN  R2              ; VAL2 is an alias for R2
SUM     RN  R3              ; SUM  is an alias for R3
MY_CONST EQU 0x25           ; EQU names a constant

        AREA MY_ASM_PROG, CODE, READONLY
        ENTRY               ; program entry point
__main
        MOV  VAL1, #MY_CONST ; R1 = 0x25
        MOV  VAL2, #0x34     ; R2 = 0x34
        ADD  SUM, VAL1, VAL2 ; R3 = R1 + R2

HERE    B    HERE           ; infinite loop to halt
        END                 ; end of assembly file
```

Three Kinds of Line

names an address

label	*mnemonic*	*operands*	*comment*	*kind of line*
`VAL1`	`RN`	`R1`	`; alias`	**directive** — assembler only
	`MOV`	`R1, #0x25`	`; R1 = 0x25`	**instruction** — becomes machine code
`HERE`	`B`	`HERE`	`; loop forever`	**instruction** — with a label

- **Instructions** (`MOV`, `ADD`, `B`) become machine code.
- **Directives** (`RN`, `EQU`, `AREA`, `ENTRY`, `END`) are commands to the assembler. They produce no instructions.
- **Labels** (`__main`, `HERE`) name an address.

`B HERE` at the end: a bare-metal program has no OS to return to, so an infinite loop is the normal, correct way to stop.

Common Assembler Directives

Directive	Example	What it does	C equivalent
AREA	AREA prog, CODE, READONLY	code section (Flash)	—
	AREA vars, DATA, READWRITE	data section (SRAM)	—
ENTRY	ENTRY	first instruction to run	main()
END	END	end of source file	—
EQU	LED_PIN EQU 5	name a constant	#define
RN	SUM RN R3	name for a register	—
DCB/DCW/DCD	x DCD 7	store 8/16/32-bit data	uint32_t x = 7;
SPACE	buf SPACE 64	reserve bytes, no value	uint8_t buf[64];
ALIGN	ALIGN 4	next 4-byte boundary	—
EXPORT	EXPORT __main	make a label public	non-static
IMPORT	IMPORT delay	use another file's label	extern

Directives produce no instructions. Only DCB/DCW/DCD and SPACE take up memory. These are Keil (armasm) names. GNU as uses .equ, .word, .global and so on.

Tracing the Three Instructions

After executing	R1	R2	R3
`MOV R1, #0x25`	`0x25`	—	—
`MOV R2, #0x34`	`0x25`	`0x34`	—
`ADD R3, R1, R2`	`0x25`	`0x34`	`0x59`

Reading the shape of a statement

The first operand is the destination. `ADD R3, R1, R2` reads as “R3 becomes R1 plus R2.” Right to left.

Keil or GNU? Same Program, Two Assemblers

Keil armasm (this book)

```
VAL1    RN   R1
VAL2    RN   R2
SUM     RN   R3
MY_CONST EQU 0x25

        AREA prog, CODE, READONLY
        ENTRY
__main
        MOV  VAL1, #MY_CONST
        MOV  VAL2, #0x34
        ADD  SUM, VAL1, VAL2

HERE    B    HERE        ; stop
        END
```

GNU as (STM32CubeIDE)

```
        .syntax unified
        .thumb
VAL1    .req r1
VAL2    .req r2
SUM     .req r3
        .equ MY_CONST, 0x25

        .text
        .global main
        .thumb_func
main:
        MOV  VAL1, #MY_CONST
        MOV  VAL2, #0x34
        ADD  SUM, VAL1, VAL2
HERE:   B    HERE        @ stop
```

Why it matters

The instructions are identical. Only directives, labels and comments differ. The book uses Keil syntax. STM32CubeIDE uses GCC, so it expects GNU syntax. Paste book code into CubeIDE unchanged, and it will not assemble.

Keil to GNU: A Translation Table

Purpose	Keil armasm	GNU as
Comment	`; text`	`@ text` or `/* text */`
Label	`HERE` (column 1, no colon)	`HERE:` (colon required)
Code / data section	`AREA p, CODE, READONLY` / `DATA, READWRITE`	`.text` / `.data`
Constant	`X EQU 5`	`.equ X, 5`
Register name	`SUM RN R3`	`SUM .req r3`
Data	`DCB` / `DCW` / `DCD`	`.byte` / `.hword` / `.word`
Reserve bytes	`SPACE 64`	`.space 64`
Align	`ALIGN 4`	`.balign 4`
Make public	`EXPORT f`	`.global f` + `.thumb_func`
Use external	`IMPORT f`	nothing needed
Entry point	`ENTRY`	none: the startup file calls `main`
End of file	`END`	`.end` (optional)
Instruction set	Thumb by default	`.syntax unified` + `.thumb`

Two traps:

- On ARM, GNU `.align 2` means $2^2 = 4$ bytes, not 2. Use `.balign 4` to say what you mean.
- Leave out `.thumb_func`, and a call from C can jump to the wrong mode and cause a HardFault.

Using the Assembler in STM32CubeIDE

1. Create an STM32 project for the NUCLEO-F446RE.
2. Right-click `Core/Src` → New → File, name it `add2.s`.
   (`.S`, capital S, runs the C preprocessor first, so `#define` works.)
3. Write the function in GNU syntax (right).
4. In `main.c`, declare and call it:
   ```
   extern int add2(int a, int b);
   r = add2(0x25, 0x34);
   ```
5. Build, then Debug. Open **Window** → **Show View** → **Disassembly** and **Registers**. Turn on **Instruction Stepping Mode** to step one instruction at a time.

```
@ add2.s
        .syntax unified
        .thumb
        .text
        .global add2
        .thumb_func
add2:                   @ a in R0, b in R1
        ADD  r0, r0, r1
        BX   lr         @ result in R0
```

Arguments arrive in R0–R3 and the result goes back in R0. Chapter 7 explains this rule (AAPCS).

Free example: `Core/Startup/startup_stm32f446retx.s` in every project is GNU assembly. Open it and read it.

Table of Contents

1 Why This Matters

2 How Much Assembly You Need

3 A First Assembly Program

4 **The Load-Store Workflow**

5 Memory Transfer Instructions

6 Optional: Signed Loads

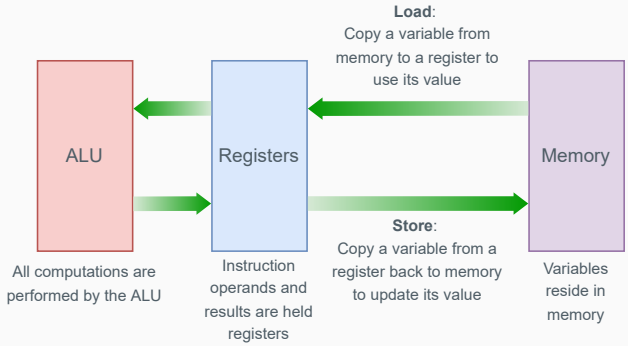
7 The Core Register Set in Practice

8 Constants: `MOV`, `MVN`, and Building an Address

9 Worked Example: Reading Your First Disassembly

10 Summary

Load, Process, Store



Every operation on a memory variable follows the same three steps. The processor cannot operate on memory directly — it only operates on registers.

One C Statement, Four Instructions

Take `c = a + b`, with all three variables in memory and `R1` already holding the base address of the block that contains them:

```
        c = a + b;
```

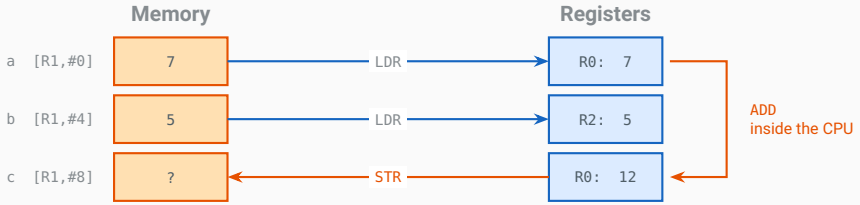
↓

```
        LDR   R0, [R1]        ; load a  (memory -> register)
        LDR   R2, [R1, #4]    ; load b  (memory -> register)
        ADD   R0, R0, R2      ; process (register + register)
        STR   R0, [R1, #8]    ; store c (register -> memory)
```

Four instructions for one assignment, and only one of them is arithmetic.

© Muním Zabidi

Where the Value Is, at Each Step



memory and registers are the only two places a value can be

Where are a, b, c right now?

After the first LDR, a exists in *two* places at once — in memory and in R0. After the ADD, c exists only inside the CPU. Lose power before the STR and the result is gone.

When the Operands Are Already in Registers

The loads and stores are not the cost of the addition. They are the cost of the operands *living in memory*. The same arithmetic on arguments already in registers:

In C

```c
int f(int a, int b) {
    return a + b;
}
```

Arguments arrive in `R0` and `R1` by the AAPCS convention. Return value leaves in `R0`.

In assembly

```
        ADD   R0, R0, R1
        BX    LR
```

Same addition. Two instructions instead of four.

Why the compiler works hard to keep values in registers

A loop that reads a global each pass is slower than one that reads it once into a local. Each memory access is an instruction the arithmetic itself does not need.

Table of Contents

1 Why This Matters

2 How Much Assembly You Need

3 A First Assembly Program

4 The Load-Store Workflow

5 **Memory Transfer Instructions**

6 Optional: Signed Loads

7 The Core Register Set in Practice

8 Constants: `MOV`, `MVN`, and Building an Address

9 Worked Example: Reading Your First Disassembly

10 Summary

The Load and Store Family

Instruction	Size	Purpose
LDR / STR	word (32-bit)	Transfer a full register
LDRB / STRB	byte (8-bit)	Transfer one byte
LDRH / STRH	halfword (16-bit)	Transfer two bytes
Less common: signed loads		
LDRSB	byte (8-bit)	Load a signed byte and sign-extend
LDRSH	halfword (16-bit)	Load a signed halfword and sign-extend

Naming convention

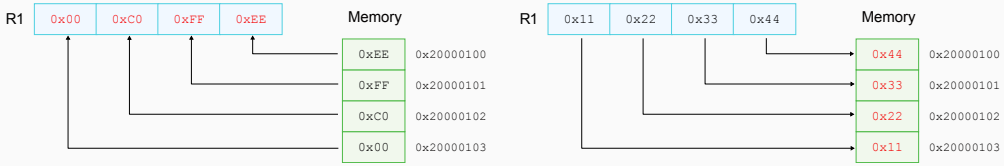
No suffix means a 32-bit word. B means byte. H means halfword. S marks a signed load. There is no signed *store* — a store just copies the low bits.

- An LDR-type instruction fetches data from memory and puts the value into the destination register, Rd.
- Syntax:

```
LDR Rd, [Rn, #offset]
```

  - `Rd`: destination register, where the loaded value is stored
  - `Rn`: base register, holding the memory address
  - `#offset`: optional offset, added to the base register to compute the memory address

Word Transfers: LDR and STR

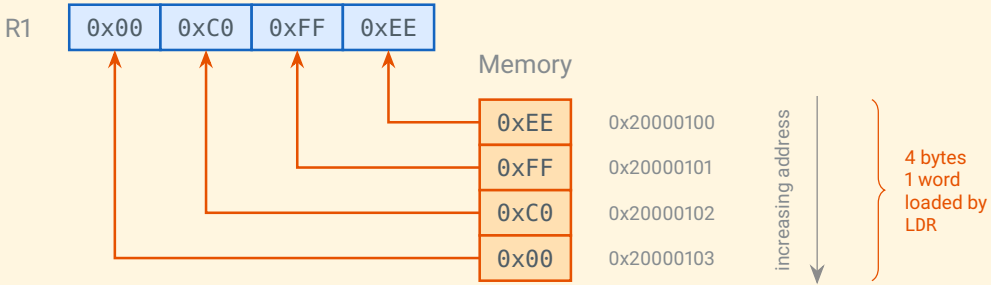


Left: `LDR R1, [R0]` with `R0 = 0x20000100` and memory bytes `EE FF C0 00`. Little-endian: the lowest-address byte becomes the least-significant byte, giving `R1 = 0x00C0FFEE`.

Right: `STR R1, [R0]` with `R1 = 0x11223344`. The least-significant byte (`0x44`) goes to the lowest memory address. The same rule, in reverse.

LDR: Loading a Word from Memory (Little-Endian)

```
LDR  R1, [R0]    ; load word
```

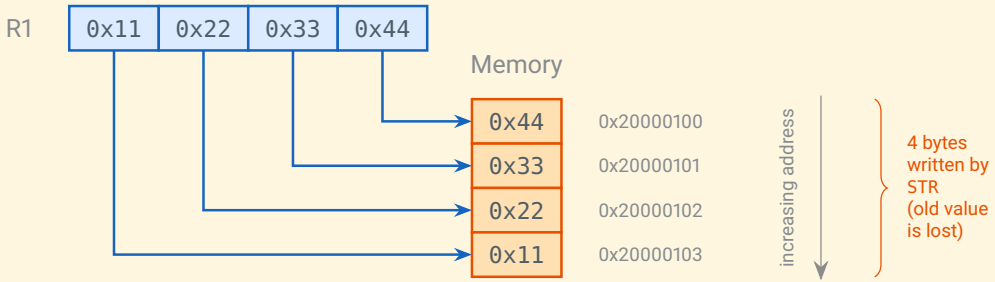


- Assume R0 = 0x20000100, and the memory word at 0x20000100 holds 0x00C0FFEE.
- After `LDR R1, [R0]` runs, the destination register R1 holds 0x00C0FFEE.

STR: Storing a Word to Memory

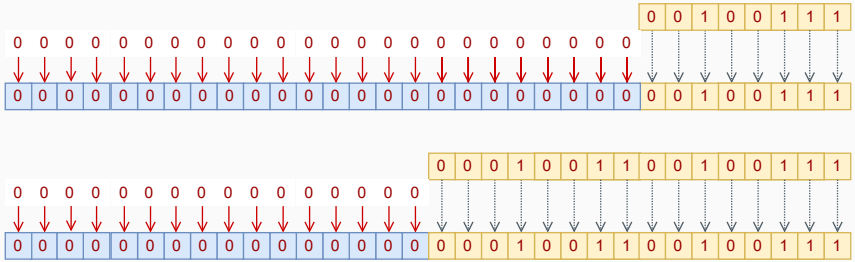
```
STR   R1, [R0]    ; store 32-bit word
```

Before: R1 = 0x11223344, [0x20000100] = 0x00C0FFEE

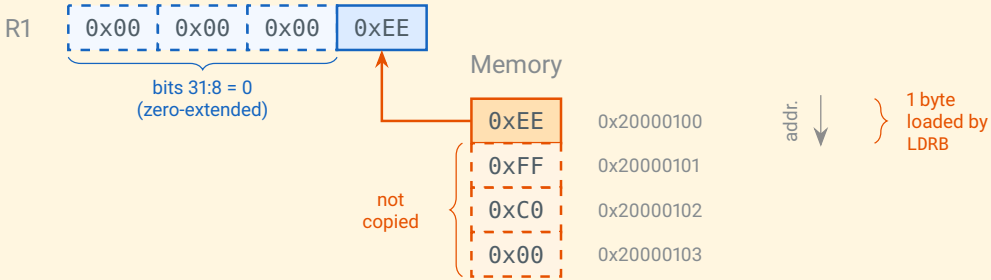


Byte and Halfword Loads: Zero Extension

The destination register is always 32 bits wide. When fewer bytes are loaded, the processor must decide what to put in the unused upper bits. Ordinary byte and halfword loads fill them with zeros.



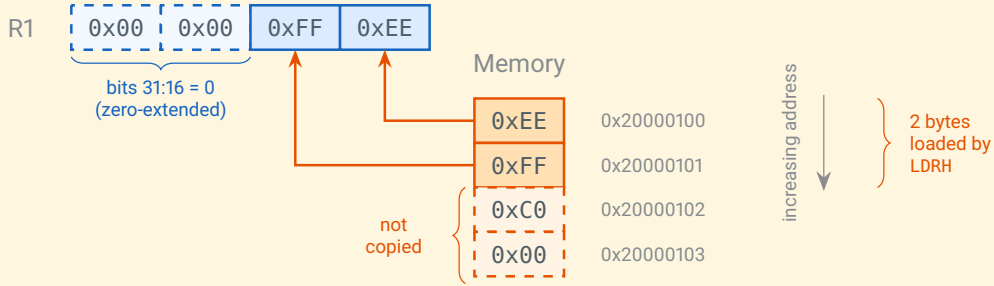
LDRB fills the upper 24 bits with zero; LDRH fills the upper 16 bits with zero.



- Assume R0 = 0x20000100, and the memory byte at 0x20000100 holds 0xEE.
- After `LDRB R1, [R0]` runs, the lowest byte of R1 holds 0xEE, and the remaining bytes are cleared.

LDRH Instruction

```
LDRH  R1, [R0]    ; load halfword
```



- Assume R0 = 0x20000100, and the memory halfword at 0x20000100 holds 0xFFEE.
- After `LDRH R1, [R0]` runs, the lower halfword of R1 holds 0xFFEE, and the top halfword is cleared.

Byte and Halfword Stores

The store side is symmetric and simpler. `STRB` writes only the low 8 bits of the register; `STRH` writes only the low 16 bits. The upper bits are ignored.

```
        LDR  R0, =0x11223344
        STRB R0, [R1]
```

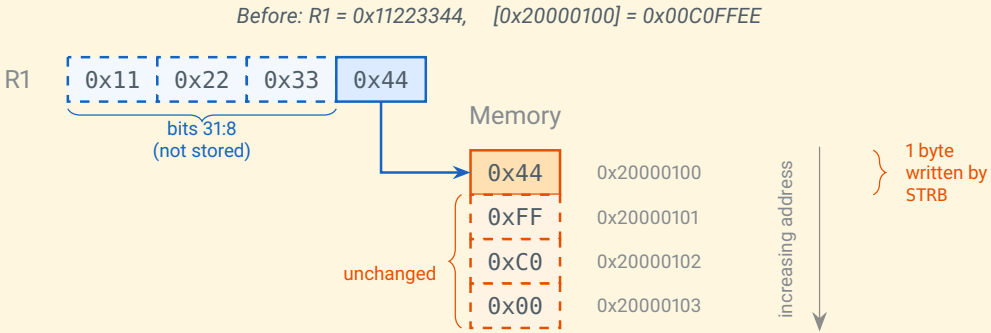
Stores only `0x44`. The upper 24 bits are ignored.

```
        LDR  R0, =0x12345678
        STRH R0, [R1]
```

Stores only `0x5678`. The upper 16 bits are ignored.

STRB: Storing a Byte to Memory

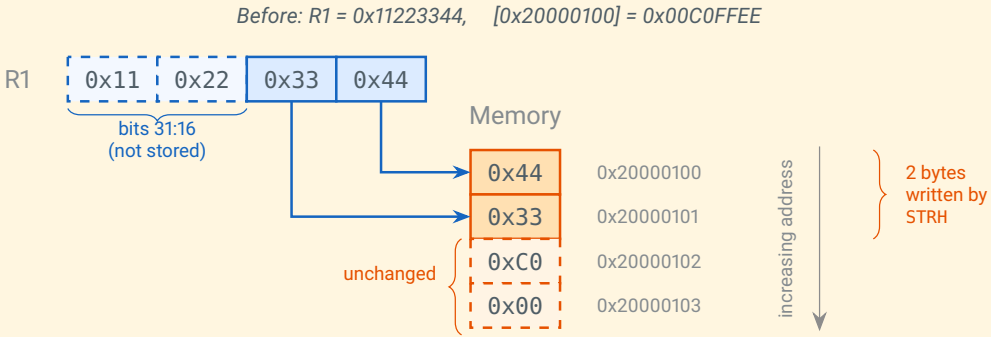
```
STRB R1, [R0]
```



- After `STRB R1, [R0]` runs, the memory byte at 0x20000100 holds 0x44.

STRH: Storing a Halfword to Memory

```
STRH R1, [R0]
```



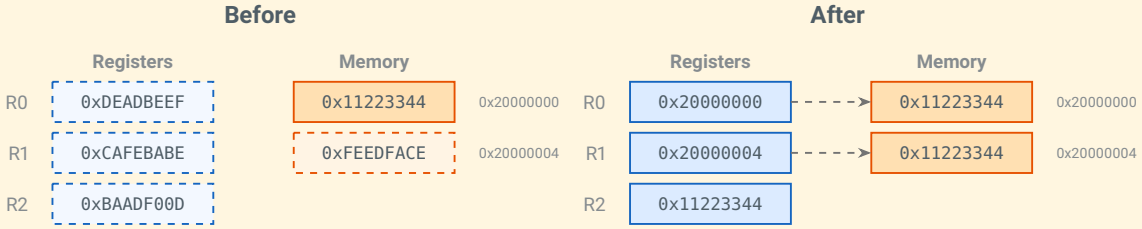
- Assume R0 = 0x20000100, R1 = 0x11223344.
- After `STRH R1, [R0]` runs, the memory halfword at 0x20000100 holds 0x3344.

Common Memory Access Instructions

Instruction	Size	Extension	Operation
`LDR Rd, [Rn]`	32-bit word	None	$Rd = [Rn]$
`STR Rt, [Rn]`	32-bit word	None	$[Rn] = Rt$
`LDRB Rd, [Rn]`	8-bit byte	Zero-extend	$Rd = \text{ZeroExtend}([Rn]_{byte})$
`STRB Rt, [Rn]`	8-bit byte	None	$[Rn]_{byte} = Rt[7:0]$
`LDRH Rd, [Rn]`	16-bit halfword	Zero-extend	$Rd = \text{ZeroExtend}([Rn]_{half})$
`STRH Rt, [Rn]`	16-bit halfword	None	$[Rn]_{half} = Rt[15:0]$

Memory Copy Example

```
LDR R0, =0x20000000
LDR R1, =0x20000004
LDR R2, [R0]
STR R2, [R1]
```



dashed arrow = address used for addressing

Table of Contents

1 Why This Matters

2 How Much Assembly You Need

3 A First Assembly Program

4 The Load-Store Workflow

5 Memory Transfer Instructions

6 **Optional: Signed Loads**

7 The Core Register Set in Practice

8 Constants: `MOV`, `MVN`, and Building an Address

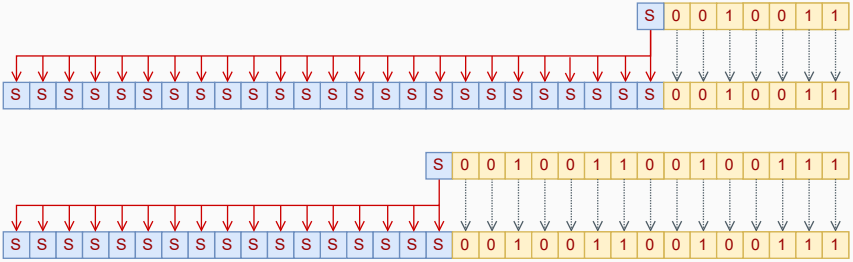
9 Worked Example: Reading Your First Disassembly

10 Summary

Signed Loads: LDRSB and LDRSH

Many embedded programs never need these. Peripheral registers, status bits, and addresses are naturally unsigned.

ARM still provides them. Instead of filling the upper bits with zeros, a signed load copies the source's sign bit into the upper bits. This is called *sign extension*.



The Same Bits, Different Numbers

Suppose memory contains the byte `0x8C`. Its binary form is `10001100`. Bit 7 is 1, so this byte represents the signed value −116.

Instruction	R0 after	Meaning
`LDRB R0, [R1]`	`0x0000008C`	decimal 140
`LDRSB R0, [R1]`	`0xFFFFFF8C`	decimal −116

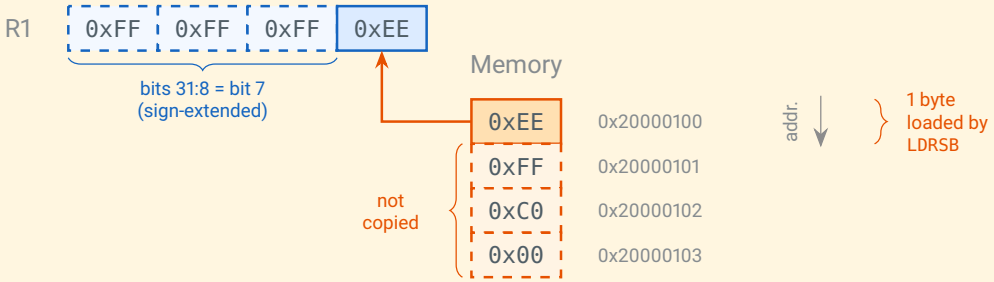
The same eight memory bits can mean different numbers, depending on which load you use.

If the sign bit is zero, both instructions give the same result. Loading `0x4F` yields `0x0000004F` either way.

C connection

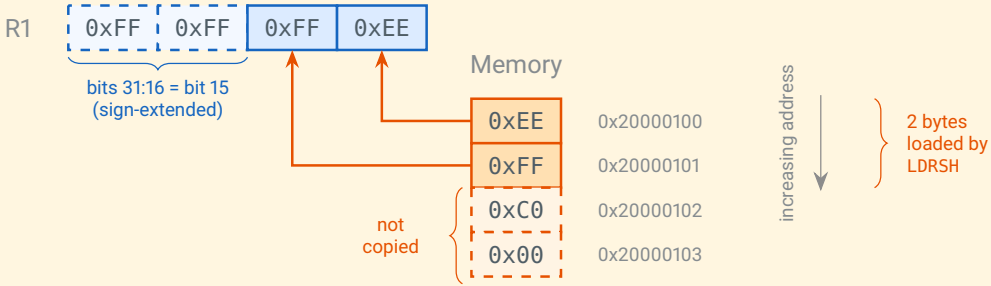
A C compiler picks `LDRB` for `uint8_t` and `LDRSB` for `int8_t`. The C type decides which instruction appears.

```
LDRSB R1, [R0]    ; load signed byte
```



- Assume R0 = 0x20000100, and the memory byte at 0x20000100 holds 0xEE.
- After `LDRSB R1, [R0]` runs, the lowest byte of R1 holds 0xEE, and the remaining bytes hold the sign bit of the loaded byte.

```
LDRSH R1, [R0]    ; load signed halfword
```



- Assume R0 = 0x20000100, and the memory halfword at 0x20000100 holds 0xFFEE.
- After `LDRSH R1, [R0]` runs, the lower halfword of R1 holds 0xFFEE, and the top halfword holds the sign bit of the loaded halfword.

`R1 = 0x20000000`. Memory there holds the bytes `EE FF C0 00` (lowest address first). Little-endian.

```
        LDR   R0, [R1]
        LDRB  R2, [R1]
        LDRSB R3, [R1]
```

Your prediction

- What is R0?
- What is R2?
- What is R3?

Predict the Register Zero extension or sign extension?

`R1 = 0x20000000`. Memory there holds the bytes `EE FF C0 00` (lowest address first). Little-endian.

```
        LDR   R0, [R1]
        LDRB  R2, [R1]
        LDRSB R3, [R1]
```

Your prediction

- What is R0?
- What is R2?
- What is R3?

The answer

`R0 = 0x00C0FFEE`: the whole word, little-endian. `R2 = 0x000000EE`: one byte, zero-extended. `R3 = 0xFFFFFFEE`: the same byte, sign-extended, because bit 7 of `0xEE` is 1.

Table of Contents

1 Why This Matters

2 How Much Assembly You Need

3 A First Assembly Program

4 The Load-Store Workflow

5 Memory Transfer Instructions

6 Optional: Signed Loads

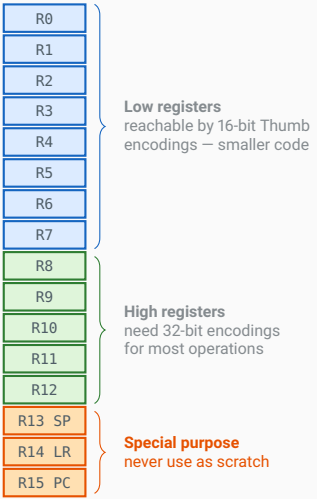
7 **The Core Register Set in Practice**

8 Constants: `MOV`, `MVN`, and Building an Address

9 Worked Example: Reading Your First Disassembly

10 Summary

Low and High Registers



Preferring `R0`–`R7` is not just a style choice. It changes the size of the compiled program.

Seen in a disassembly

```
ADDS R0, R1, R2 — 16-bit Thumb
ADDS R8, R9, R10 — 32-bit Thumb-2
```

Same operation, but double the Flash footprint.

Writing to `R15` causes an immediate branch — occasionally useful, usually a bug.

Special Registers You Will See

Three registers mean more than “a place to put a number.” A disassembly window shows them by name.

Register	Shown as	Role
R13	SP	Stack pointer — address of the last item pushed
R14	LR	Link register — BL writes the return address here
R15	PC	Program counter — writing it is a branch

The N/Z/C/V flags live in the **APSR**, part of the combined xPSR. Often shown as PSR. Chapter 7 returns to those flags.

Prologue and epilogue

```
        PUSH {LR}     ; prologue: save the return address
        ...           ; the function body
        POP  {PC}     ; epilogue: return by loading PC directly
```

`POP {PC}` *is* the return. Recognising this pair on sight is one of the most useful disassembly skills.

A Preview: Register Usage Conventions

Two disassembly listings for the same C source almost never disagree about *which* registers carry arguments and results. That is not an accident — it is the AAPCS.

Registers	Role
`R0–R3`	First four arguments in; `R0` carries the return value out
`R4–R11`	*Callee-saved* — a function may use them, but must restore them

This is why `f(a,b)` from a few slides back could add `R0` and `R1` with no register setup — the arguments were already there.

Chapter 7 builds the full convention. Appendix C tabulates it.

© Munim Zabidi

Table of Contents

1 Why This Matters

2 How Much Assembly You Need

3 A First Assembly Program

4 The Load-Store Workflow

5 Memory Transfer Instructions

6 Optional: Signed Loads

7 The Core Register Set in Practice

8 **Constants: `MOV`, `MVN`, and Building an Address**

9 Worked Example: Reading Your First Disassembly

10 Summary

Two instructions that never touch memory.

MOV Rd, Operand2 copies its second operand into Rd — an encodable constant, or another register. **MVN** does the same but inverts every bit on the way.

```
        MOV R0, R1        ; R0 = R1
        MOV R0, #25       ; R0 = 25
        MVN R0, #0        ; R0 = 0xFFFFFFFF
        MVN R0, R1        ; R0 = ~R1  (one's complement)
```

The encoding limit

MOV accepts only a restricted 12-bit encoding — an 8-bit value plus a rotation. So `MOV R0, #0xFF000000` is encodable; `MOV R0, #0x12345678` is not. Every peripheral address falls in the second category.

© Muním Zabidi

Loading a Large Constant: `LDR Rd, =value`

The form you write by hand. It is not a real instruction — it is a pseudo-instruction.

```
        LDR R0, =0x40020014
```

You say *what* value you want. The assembler decides *how* to build it:

- A single `MOV`, if the value happens to be encodable.
- Otherwise, a literal pool: a small block of data placed in the code section, read with a PC-relative load.

Programmer vs assembler

Write `LDR Rd, =value` and let the assembler decide. Read the other forms and know what they mean.

Building a 32-Bit Constant with MOVW/MOVT

One common way a Thumb-2 compiler builds a 32-bit constant: one halfword at a time. `MOVW` writes the low 16 bits and clears the high half. `MOVT` writes the high 16 bits, leaving the low half alone.

```
; build 0x40020014 (GPIOA->ODR), one halfword at a time
        MOVW R0, #0x0014   ; R0 = 0x00000014, high half cleared
        MOVT R0, #0x4002   ; R0 = 0x40020014, high half set, low half kept
```

R0 = 0x40020014

A pair to recognise, not one to write

`MOVW/MOVT` is a common outcome, not a rule. A compiler also picks a plain `MOV` for small constants, a shift, or a literal pool — depending on the value, optimization level, and target.

```
        MOVW R3, #0x0014
        MOVT R3, #0x4002   ; "R3 is being given a 32-bit address"
        LDR  R2, [R3]      ; "...and now something is read through it"
```

The useful reading is always the same

Three lines, one address being built, then used. The exact halves rarely matter. Seeing it as one operation, not three unrelated ones, is the whole skill.

C connection: a declaration as ordinary as `uint32_t x = 0x12345678;` can require two instructions. Nothing in the C source hints at the split. It is a property of the instruction encoding.

© Muním Zabidi

A Look Ahead: The Barrel Shifter

ARM can shift a register on its way *into* an instruction, at no extra cost — for example `ADD R0, R1, R2, LSL #2`.

We return to this feature in Chapter 6, where it becomes particularly useful for calculating addresses. It is mentioned here only so that a shifted operand in later disassembly does not come as a surprise.

Table of Contents

1 Why This Matters

2 How Much Assembly You Need

3 A First Assembly Program

4 The Load-Store Workflow

5 Memory Transfer Instructions

6 Optional: Signed Loads

7 The Core Register Set in Practice

8 Constants: `MOV`, `MVN`, and Building an Address

9 **Worked Example: Reading Your First Disassembly**

10 Summary

Your First Disassembly

The C

```
int c;
c = a + b;
```

The listing

```
LDR  R3, [R7, #4]
LDR  R2, [R7, #8]
ADD  R3, R3, R2
STR  R3, [R7, #12]
```

Reading it back

1. R7 is the frame pointer, so these are local variables at offsets 4, 8, 12.
2. Load two of them.
3. Add them.
4. Store the result in a third.

Three things to notice

Same load–process–store shape as every other example. No S suffix — nothing branches on the result. Offsets from R7 say stack locals, not globals.

One Operation, Viewed at Five Levels

Level	Representation
Physical hardware	Two register-file read ports feeding the ALU's 32-bit adder
Register	Two locals loaded, added, one result stored
C	`c = a + b;`
Assembly	`LDR R3, [R7, #4]; LDR R2, [R7, #8]; ADD R3, R3, R2; STR R3, [R7, #12]`
Machine code	Four Thumb-2 encodings stored in Flash

You can now

read a short assembly program, follow a value through the load–process–store cycle, and choose the correct load or store for any data width.

© Muním Zabidi

Table of Contents

1 Why This Matters

2 How Much Assembly You Need

3 A First Assembly Program

4 The Load-Store Workflow

5 Memory Transfer Instructions

6 Optional: Signed Loads

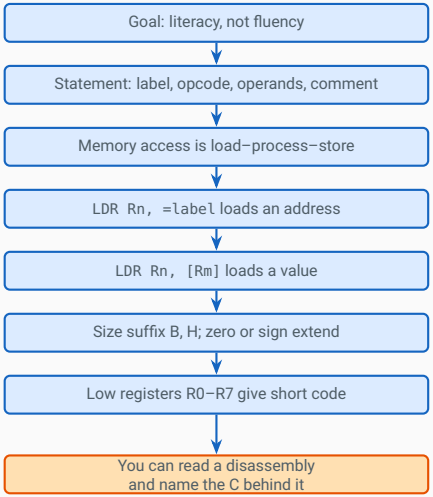
7 The Core Register Set in Practice

8 Constants: `MOV`, `MVN`, and Building an Address

9 Worked Example: Reading Your First Disassembly

10 **Summary**

Summary



Terminology covered

- Label, opcode, operand
- LDR, STR and size suffixes
- Zero and sign extension
- Low and high registers

Next: Chapter 6

Memory, Arithmetic and Bit Manipulation

How ARM calculates addresses, and how it changes single bits.

© Muním Zabidi